

# Fine-Tuning a 7B Math LLM for Olympiad Reasoning

A Diagnostic Study of Evaluation Methodology and OOD Generalization

Nate Hu Philip Wang William Xu Emory University, QTM 447

## Abstract

We fine-tune Qwen2.5-Math-7B-Instruct with LoRA on 72,283 filtered olympiad problems and evaluate on val\_200 ( $n=200$ ) and AIME 2024+2025 ( $n=60$ ). Three evaluation bugs produced a **12-point swing** in the measured SFT effect, turning an apparent  $-2$ -point regression into a genuine  $+10$ -point gain. Under corrected evaluation, Round 1 ( $r=64$ ,  $lr=2e-4$ ) achieves 67% on val\_200 but degrades on AIME 2024 ( $-6.7$  pts), a catastrophic-forgetting signature. A hyperparameter-only Round 2 ( $r=16$ ,  $lr=1e-5$ ) produces an asymmetric OOD result that implicates a persistent training/inference format mismatch as the dominant cause.

## 1 Setup

**Model and data.** We fine-tune Qwen/Qwen2.5-Math-7B-Instruct [4] (7B, already RLHF-aligned) with LoRA [2] adapters on all seven projection matrices. Training data is NuminaMath-CoT + TIR [3] filtered to olympiad-level sources with integer answers (72,283 train examples, 95/5 split, seed 42).

**Training format mismatch (key design flaw).** Both rounds train using a plain-text template ("Problem: {p}\n\nSolution: {s}"), while evaluation uses Qwen’s ChatML template. This mismatch is identified post-hoc as the likely dominant cause of OOD degradation.

**Hyperparameters.** Table 1 summarizes the two rounds; all else is identical (1 epoch, effective batch 32, cosine schedule).

Table 1: Training hyperparameters by round.

|               | Round 1 | Round 2 |
|---------------|---------|---------|
| LoRA rank $r$ | 64      | 16      |
| LoRA $\alpha$ | 128     | 32      |
| Learning rate | $2e-4$  | $1e-5$  |
| Precision     | fp16    | bf16    |

**Evaluation.** Greedy generation, last `\boxed{...}` extracted by regex, numeric-tolerant equality (" $5.0 \equiv 5$ "), `max_new_tokens=2048`, ChatML prompt format. All reported numbers use this corrected protocol (eval v2).

## 2 Results

### 2.1 Main Results

Table 2: Accuracy under corrected evaluation. Wilson 95% CIs in brackets.

| Benchmark | Baseline             | Rnd 1                       | Rnd 2                |
|-----------|----------------------|-----------------------------|----------------------|
| val_200   | 57.0<br>[53.6, 60.4] | <b>67.0</b><br>[63.6, 70.4] | 55.0<br>[48.0, 61.7] |
| AIME 2024 | 23.3<br>[11.8, 40.9] | 16.7<br>[7.3, 33.6]         | 3.3<br>[0.6, 16.7]   |
| AIME 2025 | 6.7<br>[1.6, 21.6]   | 6.7<br>[1.6, 21.6]          | 10.0<br>[3.5, 25.6]  |

Round 1 gains  $+10$  points on val\_200 ( $p < 0.01$ ) but loses 6.7 points on AIME 2024. Round 2’s smaller adapter reduces in-distribution specialization (val\_200 drops to 55%) while producing the asymmetric OOD pattern described in Section 3.

### 2.2 Evaluation Methodology Finding

Table 3: The 12-point methodological swing on val\_200.

| Model       | Eval v1 (buggy) | Eval v2 (fixed) |
|-------------|-----------------|-----------------|
| Baseline    | 36.0%           | 57.0%           |
| SFT Round 1 | 34.0%           | 67.0%           |
| $\Delta$    | $-2.0$ pts      | $+10.0$ pts     |

Three bugs in eval v1 collectively account for the swing: (1) `max_new_tokens=512` truncated solutions before `\boxed{}` (olympiad solutions run 600–900 tokens); (2) strict string equality rejected numerically equivalent answers like " $5.0$ " vs. " $5$ "; (3) plain-text prompt format at inference mismatched the ChatML distribution under which Qwen’s `\boxed{}` convention was trained. Inspecting 10 raw outputs surfaced all three bugs in  $<15$  minutes of free compute.

### 2.3 Failure-Mode Breakdown

Across all benchmarks, the dominant failure category is `wrong_numeric` (model produces a boxed integer but it is incorrect): 82% of failures on AIME for Round 1, 90% on AIME 2024 for Round 2. Only 1–13% of failures are format failures

(no\_box, truncated\_no\_box), indicating the bottleneck is reasoning capability rather than output formatting. Round 2 shows elevated wrong\_nonnumeric on val\_200 (12%), indicating partial regression in `\boxed{}` formatting discipline relative to Round 1.

### 3 Analysis

**Why does Round 1 degrade on AIME 2024?** Three mechanisms are consistent with the data: (M1) aggressive LR (2e-4) overwrites RLHF alignment; (M2) full-sequence loss dilutes gradient signal on solutions; (M3) training/inference format mismatch. Round 2 tests M1 in isolation by conserving the hyperparameters while leaving M2 and M3 unfixed.

#### Round 2 asymmetry rules out M1 as the primary cause.

If hyperparameter aggressiveness were dominant, Round 2 should have reduced AIME 2024 forgetting. Instead AIME 2024 collapsed further (16.7%  $\rightarrow$  3.3%). Meanwhile AIME 2025 directionally improved (6.7%  $\rightarrow$  10.0%), consistent with a less-specialized adapter that overfits the wrong format less—benefiting where the base had no prior competence, while further disrupting where it did (AIME 2024, val\_200).

This asymmetry implicates M3 as the dominant cause. A larger adapter (Round 1,  $r=64$ ) can absorb the format mismatch by force-fitting. A smaller adapter (Round 2,  $r=16$ ) cannot, and perturbs the base model’s alignment without compensating in-distribution gain. Conservative hyperparameters are appropriate only *given* a correctly formatted training pipeline.

### 4 Conclusion and Future Work

The central lesson is methodological: three evaluation bugs produced a 12-point swing in the measured SFT effect, enough to reverse the apparent sign of the intervention. Diagnostic inspection of raw outputs is the cheapest, highest-value step before any retraining decision.

On the modeling side, the structural fix for a Round 3 is straightforward: (1) replace the plain-text training format with `tokenizer.apply_chat_template`; (2) apply completion-only loss masking via `DataCollatorForCompletionOnlyLM`. Round 2’s hyperparameters ( $r=16$ ,  $\text{lr}=1\text{e}-5$ ,  $\text{bf}16$ ) are retained—they are appropriate given a correct format. Beyond retraining, inference-time tool integration (Python REPL via NuminaMath-TIR) is the most promising direction by literature precedent [1].

### References

- [1] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Minlie Huang, Nan Duan, and Weizhu Chen. Tora: A tool-integrated reasoning agent for mathematical problem solving, 2024.
- [2] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.
- [3] Jia LI, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Costa Huang, Kashif Rasul, Longhui Yu, Albert Jiang, Ziju Shen, Zihan Qin, Bin Dong, Li Zhou, Yann Fleureau, Guillaume Lample, and Stanislas Polu. Numinamath. <https://huggingface.co/AI-MO/NuminaMath-CoT>, 2024.
- [4] An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, Keming Lu, Mingfeng Xue, Runji Lin, Tianyu Liu, Xingzhang Ren, and Zhenru Zhang. Qwen2.5-math technical report: Toward mathematical expert model via self-improvement, 2024.